

CCOP 폐쇄망 1차 설치 — 현장 실행 절차서 (Scenario A: 네이티브 DB)

이 문서는 **한 줄씩 순서대로 따라 실행**하는 현장 작업서다. 개념·설계 배경은 `AIRGAP_VISIT_INSTALL_RUNBOOK.md`, 준비 상태는 `AIRGAP_PREFLIGHT_20260714.md` 참고.

전제: ① 1차(인프라)만 — GPU/자연어질의(Text2Cypher)는 2차. ② **Scenario A** — 현장 VM에 AgensGraph 16.9가 네이티브 선설치됨 (포트 5333). DB 컨테이너 안 씀. ③ 경량 DB 덤프·복원은 **DB팀 담당**(이 절차서는 앱이 그 DB에 붙는 부분만).

작성: 2026-07-14 · 대상 OS: Rocky Linux 10.1 · 번들: `ccop_bundle_p1` (dev 58bf8b1, 972MB)

읽는 법 (각 단계 구성)

- [목적]** 이 단계에서 무엇을/왜 하는지
- [실행]** 그대로 복사해 붙여넣을 명령
- [확인]** 성공 판단 기준 (이게 안 나오면 다음으로 넘어가지 말 것)
- [문제 시]** 실패했을 때 대응

채워 넣을 값 (미리 확보)

자리표시자	뜻	예/비고
<USB>	USB 볼륨명	맥 <code>/Volumes/<USB></code> , Rocky <code>/run/media/\$USER/<USB></code>
<VM_IP>	폐쇄망 VM 내부 IP	브라우저 접속·확인용
<DB_APP_PW>	앱 전용 DB 계정 비밀번호	DB팀이 앱 계정(<code>ccop</code>) 신설 시 발급 (V4)
<SECRET_KEY>	Flask 세션 키	<code>python -c "import secrets;print(secrets.token_hex(32))"</code>
<ADMIN_PW>	관리자 로그인 비밀번호	강한 값
<도메인>	내부 접속 도메인	기본 <code>ccop.cslee.internal</code> (인증서 CN과 일치)

PART 0 — 맥에서 USB 적재 (반입 전, 인터넷 무관)

번들은 이미 맥 `~/Downloads/ccop_bundle_p1/` 에 있음(무결성 145/145 검증됨).

Step 0-1. USB 볼륨명 확인

[목적] 복사 대상 경로 파악. USB는 반드시 **exFAT**(FAT32는 4GB 단일파일 제한). **[실행]**

```
ls /Volumes/
```

[확인] USB 이름이 보임 → 아래 `<USB>` 에 대입. **[문제 시]** 안 보이면 USB 재삽입. FAT32면 디스크 유틸리티에서 exFAT로 재포맷(데이터 삭제됨).

Step 0-2. 번들 복사

[목적] 빌드된 번들(이미지·rpm·소스)을 USB로 적재. **파일 복사일 뿐**이라 맥(arm64)이어도 이미지 내용(amd64)은 안 변함.

[실행]

```
cp -Rv ~/Downloads/ccop_bundle_p1 /Volumes/<USB>/
```

[확인]

```
du -sh /Volumes/<USB>/ccop_bundle_p1      # 약 972M
ls /Volumes/<USB>/ccop_bundle_p1          # images db rpms src SHA256SUMS
```

Step 0-3. 현장 문서 동봉 (인터넷 없는 현장 대비)

[목적] 절차서 PDF를 함께 반입해 현장에서 그대로 참조. **[실행]**

```
cp ~/test/coop_v1.0/deploy/field_kit/*.pdf /Volumes/<USB>/
```

[확인] AIRGAP_VISIT_INSTALL_RUNBOOK.pdf , AIRGAP_PREFLIGHT_20260714.pdf 존재.

Step 0-4. 반입 전 무결성 사전 검증 (권장)

[목적] 반입 게이트 통과 전에 USB 상 파일이 온전한지 미리 확인. [실행]

```
cd /Volumes/<USB>/ccop_bundle_p1 && shasum -a 256 -c SHA256SUMS | grep -c OK
```

[확인] 145 출력. [문제 시] 숫자가 다르면 Step 0-2 재복사 (USB 쓰기 오류 가능).

Step 0-5. 안전 분리

[실행]

```
diskutil eject /Volumes/<USB>
```

PART 1 — 현장 반입 & 무결성 (폐쇄망 VM)

Step 1-1. 매체 반입 행정

[목적] 기관 보안 절차 준수. [체크리스트 (실행 아님)] - [] 매체 반입 신청서 제출 + 사전 승인 - [] 반입 게이트 백신 검사 통과 - [] 매체 반출입 대장 기록(일련번호·용량·용도)

Step 1-2. USB → VM 복사

[목적] 번들을 VM 로컬로 옮김(설치 작업 루트). [실행]

```
lsblk # USB 장치명 확인 (예: sdb1)
ls /run/media/$USER/ # 자동 마운트 시 볼륨명 확인
sudo mkdir -p /opt/ccop_bundle
cp -r /run/media/$USER/<USB>/ccop_bundle_p1/* /opt/ccop_bundle/
```

[확인]

```
ls /opt/ccop_bundle # images db rpms src SHA256SUMS
```

[문제 시] 자동 마운트 안 되면 수동 마운트:

```
sudo mkdir -p /mnt/usb && sudo mount /dev/sdb1 /mnt/usb # lsblk 의 실제 장치명
cp -r /mnt/usb/ccop_bundle_p1/* /opt/ccop_bundle/
```

Step 1-3. 무결성 검증 (필수 관문)

[목적] 반입 과정에서 손상/변조 없었는지 확인. 여기서 OK 안 나오면 이후 전부 중단. [실행]

```
cd /opt/ccop_bundle && sha256sum -c SHA256SUMS | grep -c OK
```

[확인] 145 출력 (DB 덤프를 번들에 합쳤다면 +1). 실패 항목 확인:

```
sha256sum -c SHA256SUMS 2>&1 | grep -iv OK
```

[문제 시] 실패 파일이 있으면 재반입. (Rocky는 sha256sum, 맥은 shasum -a 256 — 명령이 다름에 주의)

PART 2 — Docker & 이미지 (오프라인 설치)

Step 2-1. Docker CE 오프라인 설치

[목적] Rocky 10 기본은 Podman이지만, 기존 compose를 쓰기 위해 반입한 Docker CE rpm 설치. [실행]

```
sudo dnf install -y /opt/ccop_bundle/rpms/*.rpm
```

[확인]

```
docker --version && docker compose version
```

[문제 시] 의존성 오류 시 rpm이 전부 반입됐는지 확인(`ls /opt/ccop_bundle/rpms | wc -l` → 141). 커널/아키텍처 불일치면 번들이 amd64인지 재확인.

Step 2-2. Docker 데몬 기동

[실행]

```
sudo systemctl enable --now docker
sudo systemctl status docker --no-pager | head -3
```

[확인] active (running). [참고] 이후 sudo 없이 쓰려면 `sudo usermod -aG docker $USER` 후 재로그인(선택).

Step 2-3. 컨테이너 이미지 load

[목적] 반입한 이미지 tar 3개를 로컬 Docker에 등록. (Scenario A는 앱·nginx만 필요하지만 agensgraph도 풀백용으로 load 해둠) [실행]

```
for t in /opt/ccop_bundle/images/*.tar; do echo "load $t"; sudo docker load -i "$t"; done
```

[확인]

```
sudo docker images | grep -E "ccop_app|agensgraph|nginx"
# ccop_app:1.0 · bitnine/agensgraph:v2.13.2 · nginx:alpine 세 줄
```

PART 3 — 소스 복원 & 설정

Step 3-1. 소스 복원

[목적] compose.nginx설정·인증서·앱 static·data 등 배포 파일 전개. **[실행]**

```
sudo mkdir -p /opt/ccop
sudo tar xzf /opt/ccop_bundle/src/ccop_test.tar.gz -C /opt/ccop
cd /opt/ccop && ls docker-compose.airgap.nativedb.yml deploy/ssl/cert.pem app/static
```

[확인] 세 경로 모두 존재. **[문제 시]** `docker-compose.airgap.nativedb.yml` 이 없으면 구버전 번들 — 최신(dev 58bf8b1) 소스로 재반입 필요.

Step 3-2. 바인드 마운트 권한 설정 (중요 — 자주 놓침)

[목적] 앱 컨테이너는 내부에서 **uid 1000(ccop)** 로 실행된다. 호스트의 `logs/` · `data/` 가 root 소유면 앱이 `api_keys.json` ·로그를 못 써서 기능 오류가 난다. **[실행]**

```
cd /opt/ccop
mkdir -p logs
sudo chown -R 1000:1000 logs data
```

[확인]

```
ls -ld logs data          # 소유자 uid 1000 (또는 이름)
```

Step 3-3. `.env` 작성 (Scenario A)

[목적] 앱이 네이티브 DB(호스트 :5333) 에 붙도록 환경변수 설정. 1차라 LLM은 비운다. **[실행]**

```
cd /opt/ccop
cp deploy/.env.airgap.phase1.template .env
vi .env
```

[입력 내용(.env 에서 이 값들만 채움)]

```
DB_NAME=tccopdb
DB_USER=ccop # DB팀이 신선했던 앱 전용 계정 (슈퍼유저 hlucyber 금지)
DB_PASSWORD=<DB_APP_Pw>
DB_HOST=host.docker.internal # 컨테이너 → 호스트 네이티브 DB
DB_PORT=5333
DEFAULT_GRAPH_PATH=tccop_graph_v6

# LLM - 1차: 반드시 비워둬 (SLLM_*, OPENAI_API_KEY 미설정)
FLASK_ENV=production
SECRET_KEY=<SECRET_KEY>
ADMIN_PASSWORD=<ADMIN_Pw>
LLM_API_KEY= # 외부 조회 API 미사용이면 공란(fail-closed)
CORS_ORIGINS=https://<도메인>,http://localhost
```

[확인]

```
grep -E "DB_HOST|DB_PORT|SLLM|OPENAI" .env
# DB_HOST=host.docker.internal / DB_PORT=5333 / SLLM·OPENAI 는 값 없음 (또는 주석)
```

Step 3-4. 인증서 확인 (nginx HTTPS)

[목적] nginx가 참조하는 `deploy/ssl/cert.pem` · `key.pem` 존재 확인(번들에 포함됨). **[실행]**

```
ls -l /opt/ccop/deploy/ssl/cert.pem /opt/ccop/deploy/ssl/key.pem
```

[문제 시] 없으면 현장 생성:

```
bash /opt/ccop/scripts/gen_selfsigned_cert.sh <도메인> 3650
```

Step 3-5. 도메인 반영 (선택)

[목적] nginx `server_name` 을 실제 내부 도메인으로. (IP로만 접속하면 생략 가능) **[실행]**

```
sed -n '26p' /opt/ccop/deploy/nginx.cslee.conf # 현재 server_name 확인
# 필요 시: sudo vi 로 server_name 을 <도메인> 으로 수정
```

PART 4 — 네이티브 DB 연결 확인 (DB팀 복원분)

경량 덤프의 복원 자체는 DB팀이 수행(포트 5333, tccopdb). 이 파트는 "앱이 그 DB에 닿는가"만 검증. AgensGraph 2.13→16.9 복원 호환은 V3 검증 항목.

Step 4-1. 네이티브 DB 가동 확인

[실행]

```
pg_isready -h 127.0.0.1 -p 5333 || sudo -u hlucyber pg_ctl status
```

[확인] `accepting connections`. [문제 시] DB팀에 기동 요청(`pg_ctl start`).

Step 4-2. 앱 계정 접속 + 그래프 확인

[목적] 앱 계정으로 tccopdb 접속 + 대상 그래프 존재 확인(= 데이터 복원 완료 여부). [실행]

```
psql -h 127.0.0.1 -p 5333 -U ccop -d tccopdb -c "SELECT 1;"
psql -h 127.0.0.1 -p 5333 -U ccop -d tccopdb -c "SET graph_path=tccop_graph_v6; MATCH (n) RETURN count(n);"
```

[확인] 첫 명령 `1`, 둘째 명령 노드 수(예: 149) 반환. [문제 시] 그래프 미존재/오류 = V3(복원 호환) 실패 신호 → DB팀과 플랜 B(원천 CSV → 앱 ETL 재적재) 협의.

Step 4-3. 컨테이너 → 호스트 DB 경로 확인 (V5)

[목적] 앱 컨테이너가 `host.docker.internal:5333` 으로 호스트 DB에 닿는지(방화벽·pg_hba). 앱 기동 전 미리 점검.

[실행] (임시 컨테이너로 확인)

```
sudo docker run --rm --add-host=host.docker.internal:host-gateway nginx:alpine \
  sh -c "nc -zv host.docker.internal 5333"
```

[확인] `open` / `succeeded`. [문제 시] 실패면 DBA에 요청: `postgresql.conf` 의 `listen_addresses='*'` (또는 docker 브리지 IP 포함) + `pg_hba.conf` 에 `host tccopdb ccop 172.17.0.0/16 md5` 추가 후 `pg_ctl reload`.

PART 5 — 기동 & 검증

Step 5-1. app + nginx 기동

[목적] Scenario A compose로 앱·nginx 기동(DB 컨테이너 없음). [실행]

```
cd /opt/ccop
sudo docker compose -f docker-compose.airgap.nativedb.yml up -d
```

[확인]

```
sudo docker compose -f docker-compose.airgap.nativedb.yml ps
# ccop_app, ccop_nginx 두 개 - app 은 (healthy)
```

[문제 시] `logs` 확인: `sudo docker logs ccop_app --tail 40`.

Step 5-2. 앱 헬스체크

[실행]

```
curl -s http://localhost:5001/api/v1/health
```

[확인] `{"status": "healthy"}` 계열 응답.

Step 5-3. HTTPS(nginx) 확인

[실행]

```
curl -sk https://localhost/api/v1/health
```

[확인] 동일 healthy 응답 (자체서명이라 `-k` 필수).

Step 5-4. DB 연동 확인 — V1 (핵심)

[목적] 앱↔AgensGraph 16.9 실연동. 그래프 목록이 **실데이터**로 나와야 함. [실행]

```
curl -sk https://localhost/api/v1/graph/list
```

[확인] `tccop_graph_v6` 등 실그래프 반환(빈 배열 아님). [문제 시] 빈 값/오류 → Step 4-2 재확인, `.env` 의 `DB_HOST/PORT/USER/PW`, `docker logs ccop_app`.

Step 5-5. UI 기능 검증 (브라우저, 내부망)

[목적] 1차 범위 기능 실동작 확인. [실행 (브라우저)] `https://<VM_IP>/` 접속 후: - [] UI 로딩 (다크 워크벤치) - [] 그래프 검색 → 노드 표시 - [] 노드 우클릭 → 확장(1-hop / N-depth) - [] 두 노드 경로 분석 - [] 직접 Cypher 입력 실행 → 결과 (← V1 문법 호환 최종 확인) - [] 모델러에서 수동 노드/엣지 생성 - [] (선택) 법률 RAG 적재 후 조회 — 아래 Step 5-6 - [] 자연어 질의창은 **의도적 미동작**(2차 예정 — 운영자 안내)

Step 5-6. (선택) 법률 RAG BM25 적재

[목적] 임베딩 백엔드 없이 BM25-only로 법률 근거 검색 활성화. [실행]

```
sudo docker exec ccop_app python scripts/ingest_legal_corpus.py --no-embed
curl -sk https://localhost/api/v1/legal/status
```

[확인] status에 chunks 적재 수 표시, `embedding_backend: none(bm25-only)`.

Step 5-7. 외부 호출 0건 확인 (폐쇄망 완결성)

[목적] 전 과정이 폐쇄망 내부에서만 이뤄졌는지(외부 통신 없음) 확인. [실행]

```
sudo docker logs ccop_app 2>&1 | grep -iE "openai|api.openai|https://" | head
```

[확인] 외부 도메인 호출 로그 없음. (있으면 `.env` 에 OPENAI/SLLM 잘못 설정된 것)

→ 여기까지 OK = 1차 설치 완료. 인프라·데이터·시각화·직접Cypher 검증 끝. 2차(GPU+자연어질의)는 인프라 안정화 후.

PART 6 — 문제 대응 / 롤백

증상	원인 후보	대응
<code>docker load</code> 실패	이미지 tar 손상/아키텍처	Step 1-3 무결성 재확인, 번들 amd64 확인
app 컨테이너 (unhealthy)	DB 접속 실패, 권한	<code>docker logs ccop_app</code> , Step 3-2 권한, Step 4 DB
<code>graph/list</code> 비어있음	DB 미복원/계정/포트	Step 4-2, <code>.env</code> DB_* 교정 후 <code>up -d app</code>
nginx 기동 실패	인증서 없음/경로	Step 3-4, <code>docker logs ccop_nginx</code>
컨테이너→호스트 DB 불가	pg_hba/listen_addresses	Step 4-3 DBA 협의
<code>api_keys.json</code> 쓰기 오류	logs/data 권한	Step 3-2 <code>chown 1000:1000</code>
직접 Cypher 오류(문법)	AG16.9 비호환(V1)	쿼리 형태 확인, DB팀과 호환성 검토, 플랜 B

전체 중지/재기동

```
cd /opt/ccop
sudo docker compose -f docker-compose.airgap.nativedb.yml down      # 중지
sudo docker compose -f docker-compose.airgap.nativedb.yml up -d    # 재기동
```

앱만 재기동(.env 변경 반영)

```
sudo docker compose -f docker-compose.airgap.nativedb.yml up -d app
```

부록 A — 전체 순차 명령 (요약, 붙여넣기용)

값(`<...>`)만 채우고 위→아래로. 각 블록 후 해당 Step의 [확인] 을 반드시 통과할 것.

```

### PART 1 반입 & 무결성

sudo mkdir -p /opt/ccop_bundle
cp -r /run/media/$USER/<USB>/ccop_bundle_p1/* /opt/ccop_bundle/
cd /opt/ccop_bundle && sha256sum -c SHA256SUMS | grep -c OK          # 145


### PART 2 Docker & 이미지

sudo dnf install -y /opt/ccop_bundle/rpms/*.rpm
sudo systemctl enable --now docker && docker --version
for t in /opt/ccop_bundle/images/*.tar; do sudo docker load -i "$t"; done


### PART 3 소스 & 설정

sudo mkdir -p /opt/ccop
sudo tar xzf /opt/ccop_bundle/src/ccop_test.tar.gz -C /opt/ccop
cd /opt/ccop && mkdir -p logs && sudo chown -R 1000:1000 logs data
cp deploy/.env.airgap.phase1.template .env
vi .env          # DB_HOST=host.docker.internal, DB_PORT=5333, DB_PASSWORD/SECRET_KEY/ADMIN_PASSWORD 입력, SLLM/OP
ls deploy/ssl/cert.pem deploy/ssl/key.pem


### PART 4 네이티브 DB 확인 (DB팀 복원 완료 전제)

pg_isready -h 127.0.0.1 -p 5333
psql -h 127.0.0.1 -p 5333 -U ccop -d tccopdb -c "SET graph_path=tccop_graph_v6; MATCH (n) RETURN count(n);"


### PART 5 기동 & 검증

cd /opt/ccop

sudo docker compose -f docker-compose.airgap.nativedb.yml up -d
sudo docker compose -f docker-compose.airgap.nativedb.yml ps
curl -sk https://localhost/api/v1/health
curl -sk https://localhost/api/v1/graph/list

```

부록 B — Scenario B 폴백 (네이티브 DB 복원 불가 시, DB도 컨테이너로)

V3(2.13→16.9 복원) 실패로 네이티브 DB를 못 쓸 때만, compose를 `docker-compose.airgap.yml` (DB 컨테이너 포함)로 바꾸고 `.env`의 `DB_HOST=agensgraph` · `DB_PORT=5432` 로 변경. 상세는 메인 런북 §4.5 시나리오 B 참조. (단, 이 경우 데이터는 컨테이너 DB에 별도 복원/적재 필요)

```

cd /opt/ccop

# .env: DB_HOST=agensgraph, DB_PORT=5432 로 수정

sudo docker compose -f docker-compose.airgap.yml up -d

```